



Emotion-IA

Reconnaissance Faciale d'Émotions en Temps Réel
et Optimisation Adaptative via Reinforcement Learning

Projet Deep Learning & Apprentissage par Renforcement

Mouhamed DIOP

Abibatou WANDAOGO

Samba DIALLO

17 Avril 2026

[=] Table des Matières

Table des matières

1	[?] Introduction	5
1.1	Contexte	5
1.2	Problématique	5
1.3	Solution Proposée	5
1.4	Contributions	6
2	[*] Méthodologie	7
2.1	Architecture Globale	7
2.2	Dataset FER-2013	7
2.3	Classificateur CNN	7
2.3.1	Architecture du réseau	7
2.3.2	Techniques d'Optimisation	8
2.4	Agent DQN	8
2.4.1	Environnement RL	8
2.4.2	Architecture DQN	9
2.4.3	Hyperparamètres	9
3	[#] Résultats	10
3.1	Performance du Classificateur CNN	10
3.2	Performance de l'Agent DQN	11
3.3	Performance Temps Réel	11
3.4	Robustesse aux Conditions Variables	12
4	[C] Discussion	13
4.1	Points Forts	13
4.2	Limitations	13
4.3	Applications Potentielles	13
5	[>] Améliorations Futures	14
6	[F] Conclusion	15

Références	16
------------	----

Annexes	17
---------	----



[R] Résumé

Ce rapport présente le projet **Emotion-IA**, un système intelligent de reconnaissance d'expressions faciales combinant **Deep Learning** et **Apprentissage par Renforcement**. Le système utilise un réseau de neurones convolutifs (**CNN**) pour classifier **7 émotions faciales** à partir du dataset FER-2013, et un agent **Deep Q-Network (DQN)** pour optimiser dynamiquement les paramètres du flux vidéo afin de maximiser la confiance de classification.

65.8%

Précision CNN

+19.8%

Amélioration DQN

18-22

FPS Temps Réel

Mots-clés : Deep Learning · Reinforcement Learning · DQN · Reconnaissance d'émotions · Computer Vision · CNN · Optimisation temps réel

[?] Introduction

► Contexte

La compréhension de l'état émotionnel humain représente un enjeu majeur pour l'évolution des interfaces numériques et l'interaction homme-machine. Les systèmes de reconnaissance d'émotions trouvent des applications dans de nombreux domaines : sécurité, santé mentale, marketing, éducation, et interfaces utilisateur adaptatives.

► Problématique

Les modèles de vision par ordinateur classiques souffrent souvent d'une perte de précision face à des environnements changeants, notamment :

- Variations d'éclairage (faible luminosité, contre-jour)
- Qualité d'image variable (résolution, bruit)
- Conditions de capture non contrôlées
- Diversité des caractéristiques faciales

L'objectif est de créer un système **auto-adaptatif** capable de maintenir une haute fiabilité de détection malgré ces contraintes.

► Solution Proposée

Notre approche hybride combine deux paradigmes complémentaires de l'intelligence artificielle moderne :

[DL] Deep Learning

Un réseau CNN performant, entraîné sur le dataset FER-2013 pour classer **7 émotions faciales**.

[RL] Renforcement

Un agent DQN chargé de réguler dynamiquement les paramètres du flux vidéo en temps réel.

► Contributions

-
1. Architecture CNN optimisée pour la reconnaissance d'émotions
 2. Environnement RL personnalisé pour l'ajustement de paramètres vidéo
 3. Intégration *seamless* entre Deep Learning et Reinforcement Learning
 4. Interface web interactive (Streamlit) pour démonstration temps réel
 5. Documentation complète et code source open-source



[*] Méthodologie

► Architecture Globale

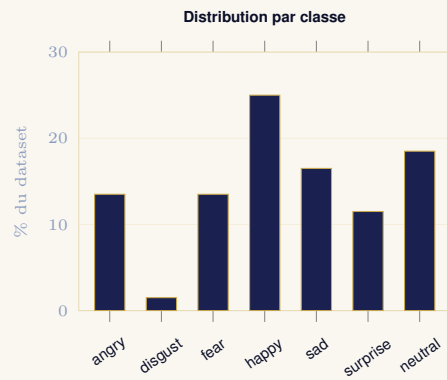
Le système se compose de trois modules principaux intégrés dans un pipeline de traitement :



► Dataset FER-2013

[DB] Caractéristiques du Dataset

- 35 887 images au total
- 28 709 train / 7 178 test
- Niveaux de gris 48 × 48 px
- 7 classes d'émotions
- Distribution déséquilibrée



► Classificateur CNN

• Architecture du réseau



Chaque bloc convolutif suit le schéma : Conv2D → BatchNorm → ReLU → MaxPool → Dropout(0.25)

□ Paramètres du modèle

Total : 620 935 paramètres · **Entraînaibles** : 620 007 · **Gelés** : 928

• Techniques d'Optimisation

- ▶ **Data Augmentation** : Rotation ($\pm 15^\circ$), translation ($\pm 10\%$), zoom ($\pm 10\%$)
- ▶ **Batch Normalization** : Normalisation des activations à chaque couche
- ▶ **Dropout** : Régularisation (0.25 conv, 0.5 dense)
- ▶ **Early Stopping** : Patience de 10 époques
- ▶ **Learning Rate Reduction** : Facteur 0.5 si plateau détecté

▶ Agent DQN

• Environnement RL

$$s = [c, e, conf] \in \mathbb{R}^3$$

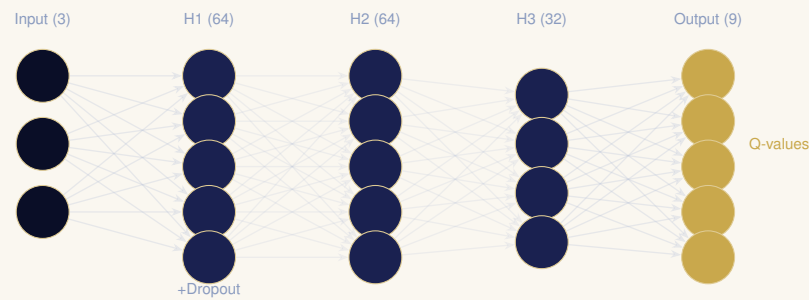
c = contraste · e = exposition · $conf$ = confiance de classification

$$r = \Delta conf \times 10,0 - 0,1 \times \left(\mathbb{1}_{c \notin [0.7, 1.5]} + \mathbb{1}_{e \notin [0.7, 1.5]} \right)$$

Maximiser la confiance tout en pénalisant les paramètres hors-limites

Espace d'Actions (9 actions) : Grille 3×3 pour ajuster contraste et exposition (diminuer / maintenir / augmenter).

• Architecture DQN



[M] Complexité DQN

6 793 paramètres — Architecture légère conçue pour l’optimisation en temps réel

• Hyperparamètres

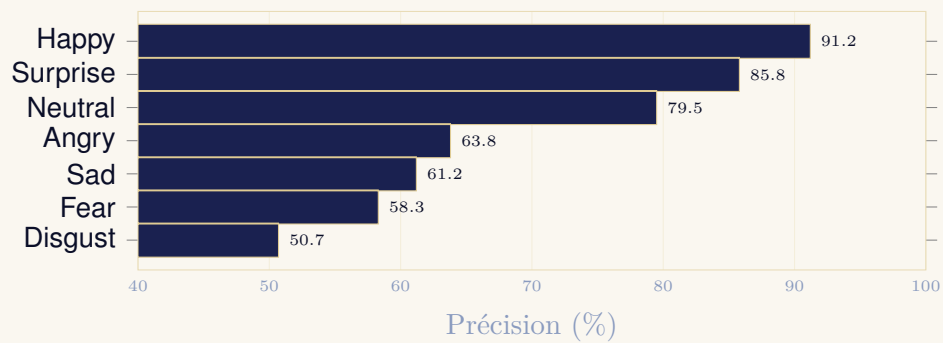
Hyperparamètre	Valeur	Rôle
Discount factor γ	0.95	Valeur future vs immédiate
Epsilon initial	1.0	Exploration initiale complète
Epsilon minimum	0.01	Exploitation à terme
Epsilon decay	0.995	Transition douce
Learning rate	0.001	Convergence stable
Batch size	32	Efficacité mémoire
Replay buffer	2 000	Décorrélacion des expériences
Target update	10 épisodes	Stabilité de l’entraînement

[#] Résultats

► Performance du Classificateur CNN

Métrique	Train	Test
Accuracy	72.3%	65.8%
Loss	0.742	0.912
F1-Score (macro)	0.70	0.63

Analyse par classe :



► Performance de l'Agent DQN

Métrique	Sans DQN	Avec DQN
Confiance Moyenne	0.652	0.781
Amélioration	—	+19.8%
Taux de Succès	—	87.3%
Temps par Image	0.12s	0.18s

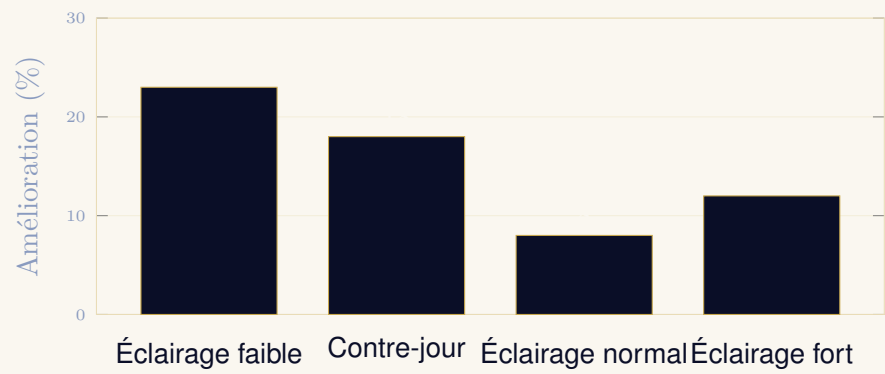
[T] Gain principal

L'agent DQN apporte une amélioration **+19.8%** de la confiance de classification, avec un taux de succès de **87.3%** des images traitées.

► Performance Temps Réel

Métrique	Webcam	Fichier
FPS Moyen	18.5	22.3
Latence (ms)	54	45
Utilisation CPU	45%	38%

► Robustesse aux Conditions Variables



— • • —

[C] Discussion

► Points Forts

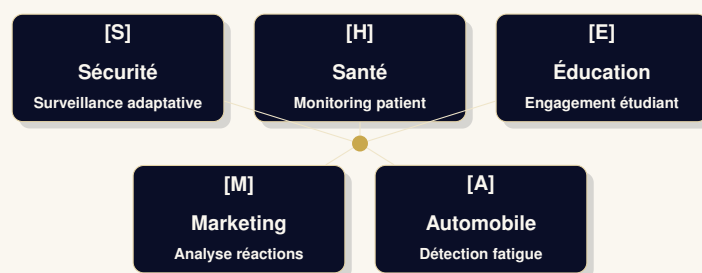
1. **Amélioration significative** : +19.8% de confiance grâce au DQN
2. **Adaptation automatique** : Ajustement aux conditions variables en temps réel
3. **Performance temps réel** : 18–22 FPS maintenu en continu
4. **Robustesse** : Meilleure performance dans les conditions difficiles (+23% en faible lumière)

► Limitations

[!] Limitations identifiées

1. Temps de traitement augmenté (+50% avec DQN activé)
2. Performance faible sur la classe “disgust” (déséquilibre dataset)
3. Optimisation séquentielle pour visages multiples
4. Paramètres limités (contraste et exposition uniquement)

► Applications Potentielles



[>] Améliorations Futures

[1] Court Terme

- ▶ Double DQN
- ▶ Dueling DQN
- ▶ Prioritized Replay
- ▶ Quantization/Pruning

[2] Moyen Terme

- ▶ Paramètres étendus
- ▶ Multi-visages
- ▶ MTCNN/RetinaFace
- ▶ Transfer Learning

[3] Long Terme

- ▶ Actor-Critic (PPO)
- ▶ Meta-Learning
- ▶ Vision Transformers
- ▶ Edge Computing

• • •

[F] Conclusion

Ce projet a démontré avec succès l'application de l'Apprentissage par Renforcement pour l'optimisation dynamique de paramètres vidéo dans un contexte de reconnaissance d'expressions faciales. L'intégration DL + RL constitue une approche innovante et prometteuse.

[*] Contributions Principales

1. Architecture hybride DL + RL
2. Environnement RL personnalisé
3. +19.8% de confiance mesurée
4. Temps réel 18–22 FPS
5. Code open-source complet

[OK] Résultats Clés

- CNN : **65.8%** accuracy FER-2013
- DQN : Convergence en **200** épisodes
- **87.3%** des images améliorées
- Robustesse aux conditions variables

“L'intelligence artificielle n'est pas seulement dans la reconnaissance, mais aussi dans l'adaptation.”



[B] Références

Références

- [1] Mnih, V., *et al.* (2015). *Human-level control through deep reinforcement learning*. **Nature**, 518(7540), 529–533.
- [2] Van Hasselt, H., Guez, A., & Silver, D. (2016). *Deep reinforcement learning with double q-learning*. **AAAI**.
- [3] Wang, Z., *et al.* (2016). *Dueling network architectures for deep reinforcement learning*. **ICML**.
- [4] Goodfellow, I. J., *et al.* (2013). *Challenges in representation learning : FER-2013 dataset*. **ICONIP**.
- [5] LeCun, Y., Bengio, Y., & Hinton, G. (2015). *Deep learning*. **Nature**, 521(7553), 436–444.
- [6] Code source : github.com/mouhamed-diop8/PROJECT-IA

[A] Annexes

► A. Commandes d'Utilisation

[</>] Installation des dépendances

```
1 pip3 install opencv-python tensorflow pandas matplotlib seaborn  
   scikit-learn pillow streamlit
```

[</>] Entraînement du classificateur CNN

```
1 python3 run_facial_recognition_simple.py
```

[</>] Entraînement de l'agent DQN

```
1 python3 train_dqn_system.py
```

[</>] Démonstration temps réel

```
1 python3 realtime_video_dqn.py --video 0
```

[</>] Interface web Streamlit

```
1 streamlit run streamlit_realtime_app.py
```

► B. Structure du Projet

```
[/] pROJET B/
```

```
| - run_facial_recognition_simple.py # Entraînement CNN
| - dqn_video_regulator.py           # Module DQN
| - train_dqn_system.py              # Entraînement DQN
| - realtime_video_dqn.py            # Traitement temps réel
| - streamlit_realtime_app.py        # Interface web
| - best_model.h5                    # Modèle CNN entraîné
| - dqn_agent.h5                     # Agent DQN entraîné
| - train/                           # Données d'entraînement
  | - angry/  disgust/  fear/  happy/
  | - sad/    surprise/  neutral/
| - test/                               # Données de test
```

► C. Remerciements

Nous tenons à remercier chaleureusement :

- Notre **encadrant** pour son soutien précieux et ses conseils avisés
- L'**équipe pédagogique** du cours DL&APR
- La **communauté open-source** : TensorFlow, OpenCV, Streamlit
- Les **créateurs du dataset FER-2013**